block(width: 100%, outset: (x: 25mm), fill: navy, inset: (top: 18mm, bottom: 18mm),)[

TP 02

Programmation du noyau Linux

Appels système et modules noyau

AUTEUR
Lionel Auroux

DATE
2026

DURÉE
5h

]

Sommaire

Objetifs 3
L'appel système `hello` 3
L'appel système `uptime` 4
LKM `version` 5

Objetifs

L'objectif de cet atelier est de réaliser des premiers codes en kernel mode:

- Ajout d'un appel système
- Un premier Linux Kernel Module



Fig. 1. – width:400px

L'appel système `hello`

Écrivez un appel système qui, lorsqu'il est invoqué depuis l'espace utilisateur, affiche `"hello, world!"` dans le journal du noyau.

Testez votre appel système dans votre image LFS avec un programme utilisateur appelant simplement `syscall` avec le bon numéro de syscall.

```c
// my_userland_test_syscall.c
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

#define MY_SYSCALL_NUMBER       ?? // dépend de votre syscall

int main()
{
    syscall(MY_SYSCALL_NUMBER);
    return 0;
}
```

Vous apprendrez :

- Comment ajouter un nouvel appel système à Linux.
- Comment utiliser votre appel système depuis l'espace utilisateur.

Voir aussi :

- Anatomy of a system call
- syscall(2)

L'appel système `uptime`

L'objectif de ce projet est d'ajouter l'appel système `uptime` au noyau Linux.

Vous devez :

- Modifier les sources et ajouter le syscall `uptime()`.
- Rendre votre appel système optionnel via kconfig.
- Configurer, compiler et exécuter votre noyau.

Voici la spécification du syscall `uptime()` :

```
code

NOM
       uptime - obtenir le temps de fonctionnement en secondes

SYNOPSIS
       time_t uptime(time_t *t);

DESCRIPTION
       uptime() retourne le temps de fonctionnement de la machine en nombre de
secondes.

       Si t est non-NULL, la valeur de retour est également stockée dans la
mémoire
       pointée par t.

VALEUR DE RETOUR
       En cas de succès, le temps de fonctionnement en secondes.
       En cas d'erreur, ((time_t) -1) est retourné.
```

Indice : `uptime` lit les mêmes données que celles disponibles dans `/proc/uptime`.

Vous apprendrez :

- Comment ajouter un nouvel appel système à Linux.
- Comment utiliser votre appel système depuis l'espace utilisateur.
- Comment transmettre des valeurs du noyau vers l'espace utilisateur.
- Comment utiliser Kconfig pour compiler le syscall `uptime` de façon optionnelle.

LKM `version`

L'objectif de ce projet est de créer un module noyau chargeable simple.

Vous devez :

- Créer un module hors-arbre (*out-of-tree*).
- Créer un programme de test.

```shell
version-login_x
├── Makefile
├── test.c
├── version.c
└── version.h
```

Le module doit effectuer les actions suivantes :

- Enregistrer un nouveau périphérique `misc` : `version`. On verra apparaître `/dev/version`.
- La lecture sur le périphérique retourne la version courante du noyau, telle que lue dans `/proc/sys/kernel/version`.
- L'écriture sur le périphérique modifie la version courante du noyau.
- Le périphérique supporte deux IOCTL :
  - `VERSION_MODIFIED` : indique si la version a été modifiée par rapport à l'originale.
  - `VERSION_RESET` : remet la version originale du noyau.

Vous apprendrez :

- Comment construire un module Linux chargeable hors-arbre.
- Comment charger un module.
- Comment enregistrer un périphérique misc.
- Comment définir les opérations `read`, `write` et `ioctl` sur un périphérique.